

Taller Circuit Breaker

1. Experimento

Título del experimento	Circuit breaker en el sistema de control térmico
Propósito del experimento	Comprobar si la táctica "fallar con gracia" (a través de un circuit breaker) mejora o no la disponibilidad del sistema de control térmico.
Resultados esperados	Cantidad de peticiones satisfechas en condiciones normales y de falla.
Infraestructura computacional requerida	- Instancias de MVs en GCP. - Script IaC para despliegue. - Computador personal para acceder a los servicios web.
ASRs involucrados	- Yo como CTO de Gerencia del Campus, dado que el sistema se está recuperando de una falla, quiero que los usuarios puedan seguir accediendo a los servicios sin recibir un mensaje de error del servidor, esto debe ocurrir un 99% de las veces. - Yo como usuario del sistema de Gerencia del Campus, dado que el sistema se está recuperando de una falla, quiero ser informado de la falla en la página principal del sitio web. Este mensaje debe ser mostrado máximo una hora después del inicio de la falla.
Descripción del experimento	En este experimento se implementa un Circuit Breaker en un sistema que tiene dos servicios (alarmas y monitoring app), donde uno de los servicios tiene varias réplicas (alarmas). Se usará Kong para configurar el Circuit-Breaker y Django para los servicios web. Se probará el Circuit Breaker en condiciones normales y de falla (inhabilitando varias instancias del servicio de alarmas). Adicionalmente, se replicará el servicio de monitoring app para poder aplicar un Circuit Breaker que mejore su disponibilidad.

1.1. Estilos de arquitectura

Estilos de Arquitectura asociados	Análisis (Atributos de calidad que favorece y desfavorece)
Monolito	• Favorece latencia y mantenibilidad. • Desfavorece escalabilidad y disponibilidad.
Cliente-servidor	• Favorece seguridad (el control de acceso a los datos es centralizado). • Desfavorece escalabilidad y disponibilidad.
Capas - MVC	• Favorece la mantenibilidad y la flexibilidad del sistema. • Desfavorece latencia y aumenta la complejidad.

1.2. Tácticas

Tácticas de Arquitectura asociadas	Descripción
Load Balancer + Stateless Nodes	Esta táctica de manejo de recursos beneficia el desempeño y la escalabilidad de un sistema al tener más nodos de computación en los que se procesan las solicitudes. En este escenario la carga se reparte en todos los nodos configurados.
Fallar con gracia	Esta táctica de control de acceso a recursos consiste en que cuando un servicio falla, se pueda responder de la mejor manera posible. Beneficia la disponibilidad al tener más control de los fallos del sistema y poder decidir cómo se muestra a los interesados (usuarios finales y equipo de desarrollo). El patrón usado para aplicar esta táctica en este laboratorio es Circuit Breaker .

Health checks	Los health checks son una táctica que beneficia la disponibilidad al detectar errores en los servicios del sistema. Consiste en enviar mensajes periódicamente a un servicio para comprobar que esté en línea, si el servicio no responde, se entiende que el servicio está caído.
----------------------	--

Circuit Breaker

Generalmente los sistemas de software están compuestos de varios servicios o componentes dispersos entre diferentes nodos de ejecución a lo largo de la red. Esta distribución entre diferentes nodos agrega comunicaciones a través de peticiones que pueden presentar fallos, sobrecargar los servicios o generar tiempos de espera altos mientras recursos críticos responden, por lo que se hace necesario buscar estrategias (tácticas) que **mejoren la disponibilidad del sistema**, por ejemplo, implementar un Circuit Breaker que permita **enmascarar las fallas** y permitir **fallar con gracia** de una aplicación.

El circuit breaker enmascara el llamado a los servicios o componentes de software y monitorea si están funcionando correctamente o si están fallando. Si todos los servicios o parte de ellos están funcionando correctamente (por encima de un umbral) se responde al cliente. Una vez se está por debajo del umbral de "sanidad", el circuito se abre, y todas las peticiones retornarán un error, sin proteger así el llamado. El diagrama de secuencia que sigue describe el comportamiento del circuit breaker. La 1° petición es resuelta por el servicio proveedor (supplier) sin inconveniente. En la 2° y 3° petición al supplier (las cuales son fallidas por un problema de conexión), el circuit breaker deja de enmascarar el fallo y lo envía al cliente. Finalmente, después de la 3° petición fallida se abre el circuito lo cual puede ser útil para notificar del fallo al equipo de arquitectos a fin de que estos lo reparen.

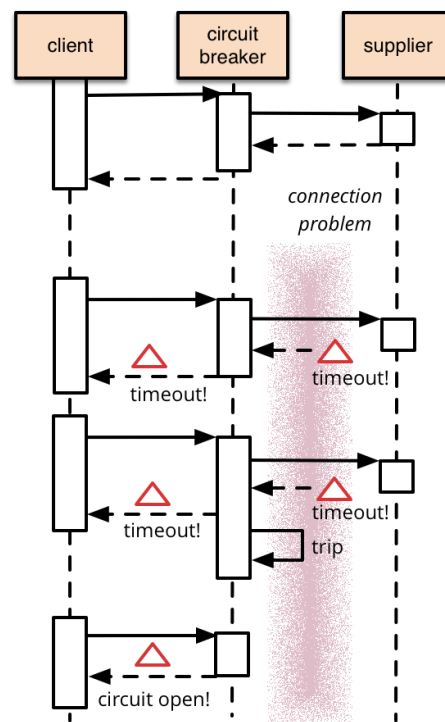


Figura 1. Diagrama de secuencia de un circuit breaker.
Tomado de [CircuitBreaker \(martinfowler.com\)](http://martinfowler.com/CircuitBreaker)

Balancedores de carga

Generalmente el ambiente en el que se ejecutan las aplicaciones no es suficiente para cumplir los requerimientos de calidad que demanda el cliente, por lo que se hace necesario buscar estrategias (tácticas) que **mejoren el desempeño del sistema**, por ejemplo, implementar un balanceador de carga en muchas ocasiones puede mejorar el **escalamiento** y **latencia** de una aplicación.

1.3. Elementos de arquitectura

1.3.1. Diagrama de despliegue

En este taller, vamos a implementar un Circuit Breaker que enmascare las fallas cuando el sistema se degrade y permita fallar con gracia para evitar peticiones fallidas al usuario. En la figura 2 se presenta la vista de despliegue, las instancias alarms-app-a, alarms-app-b y alarms-app-d contienen el mismo servicio REST desarrollado en Python + Django. Una base de datos persistirá la información y está alojada en la instancia db-django. El circuit breaker se despliega con el servicio de Kong en la instancia kong-instance y finalmente, se realizarán las pruebas de disponibilidad del sistema desde una maquina personal.

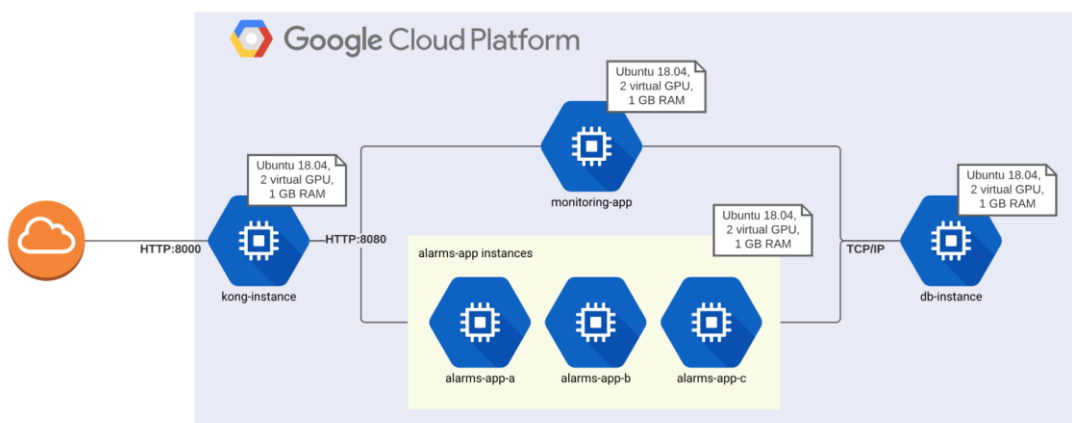


Figura 2. Vista de despliegue

1.4. Tecnologías

Tecnologías asociadas	Selección y Justificación
Frameworks	Django: Es un framework Web robusto y de rápida implementación que usa el modelo MVC.
Lenguajes de programación	Python: Es el lenguaje principal de desarrollo de Django.
Plataforma de despliegue	- GCP: La nube computacional permite escalabilidad y flexibilidad en la infraestructura. GCP es la nube patrocinadora del proyecto. - GCP Deployment Manager: Es un servicio de despliegue de infraestructura (IaC) que automatiza la creación y gestión de recursos en GCP.
Bases de datos	PostgreSQL: Es una base de datos SQL popular que favorece la disponibilidad, la latencia y la flexibilidad.
Herramientas de análisis	Navegador: Es la herramienta que permite interactuar con los servidores desplegados (web y alarmas).
Librerías	Psycopg2: Es la librería que permite la integración entre Django y PostgreSQL.

Otras	• Kong Gateway: Es un API gateway optimizado para arquitecturas distribuidas o con microservicios. • Docker: Es una tecnología de contenerización que permite la distribución y ejecución rápida de aplicaciones.
-------	---

Infraestructura como código (IaC)

Durante los laboratorios desplegados en GCP se le ha pedido crear manualmente servidores de distintos tipos en máquinas virtuales, bases de datos SQL o firewalls. En sistemas de nube más complejos, pueden ser necesarios otros servicios como almacenamiento de archivos, redes privadas, otros tipos de base de datos, tareas programadas, contenedores, entre otros. En este último caso la creación y administración de forma manual de cada recurso no es eficiente y puede ser causa de errores dentro del sistema. Por ejemplo, usted puede tomar varias horas para crear 5 máquinas, ejecutar sus servicios y puede que no funcione correctamente por una IP mal escrita o un error en el nombramiento de las máquinas. Una forma de solucionar esto es con IaC. La infraestructura como código (IaC, por sus siglas en inglés) automatiza el despliegue y facilita la administración de los recursos de un sistema.

GCP Deployment Manager (GCP DM) es una herramienta de infraestructura como código, desarrollada específicamente para Google Cloud Platform (GCP), que facilita el despliegue de infraestructura mediante:

- Lenguaje declarativo: En archivos de configuración YAML se puede describir la infraestructura de un sistema. Se especifica los recursos que se usan e incluso se configuran. GCP DM se encarga de desplegar los recursos definidos en estos archivos.
- Versionamiento y control de cambios: Ahora, con la infraestructura de un sistema descrita en archivos de configuración, es posible tener distintas versiones y se puede aprovechar los beneficios de herramientas como Git. Podría ser muy útil cuando se quiere volver a un estado anterior de la infraestructura.

En este laboratorio se usará GCP DM para desplegar la infraestructura necesaria para el laboratorio. A lo largo del laboratorio, reflexione sobre las ventajas y desventajas que trae el uso de esta tecnología. Para más información puede consultar la [documentación](#) en GCP.

2. Actividades

Este taller debe realizarse **individualmente**.

2.1. Desplegar la infraestructura usando IaC

Un requerimiento de GCP Deployment Manager para desplegar la infraestructura es un archivo de configuración YAML. Este archivo de configuración requiere de cambios para ajustarlos a su proyecto. Para realizar estos cambios se aprovecha la consola Shell y el editor del proyecto de GCP.

1. En GCP, con el proyecto seleccionado que va a trabajar, abra la consola **CloudShell** con el botón que aparece en la esquina superior derecha. En caso de que GCP le pida autorización, acéptela.



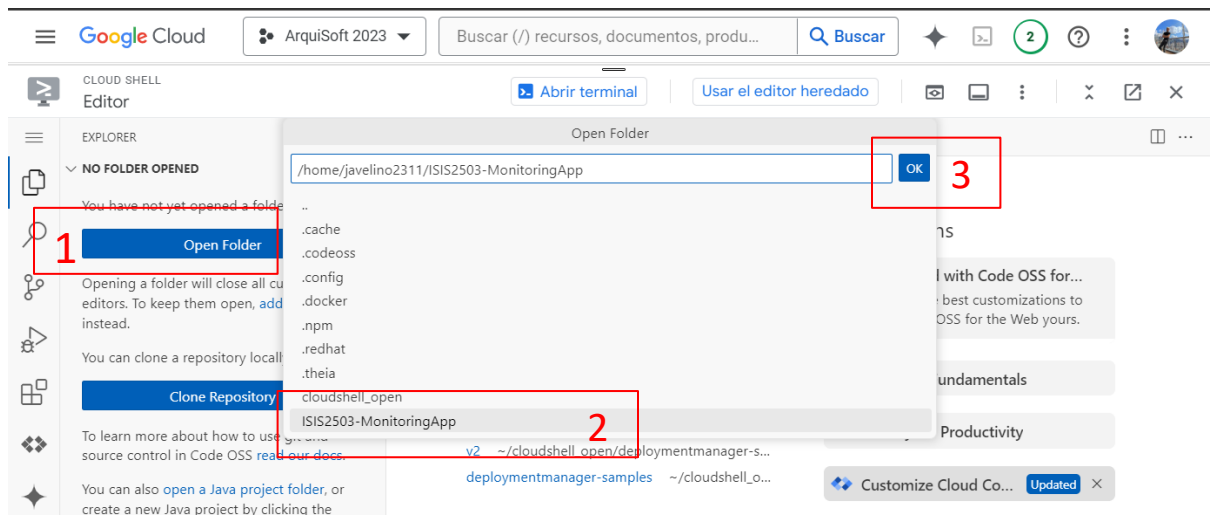
2. Con CloudShell abierto, clone el repositorio de los laboratorios y seleccione la rama "Circuit-Breaker". Puede ejecutar los siguientes comandos:

```
>>> git clone https://github.com/ISIS2503/ISIS2503-MonitoringApp.git
>>> cd ISIS2503-MonitoringApp/
>>> git checkout Circuit-Breaker
>>> git pull
```

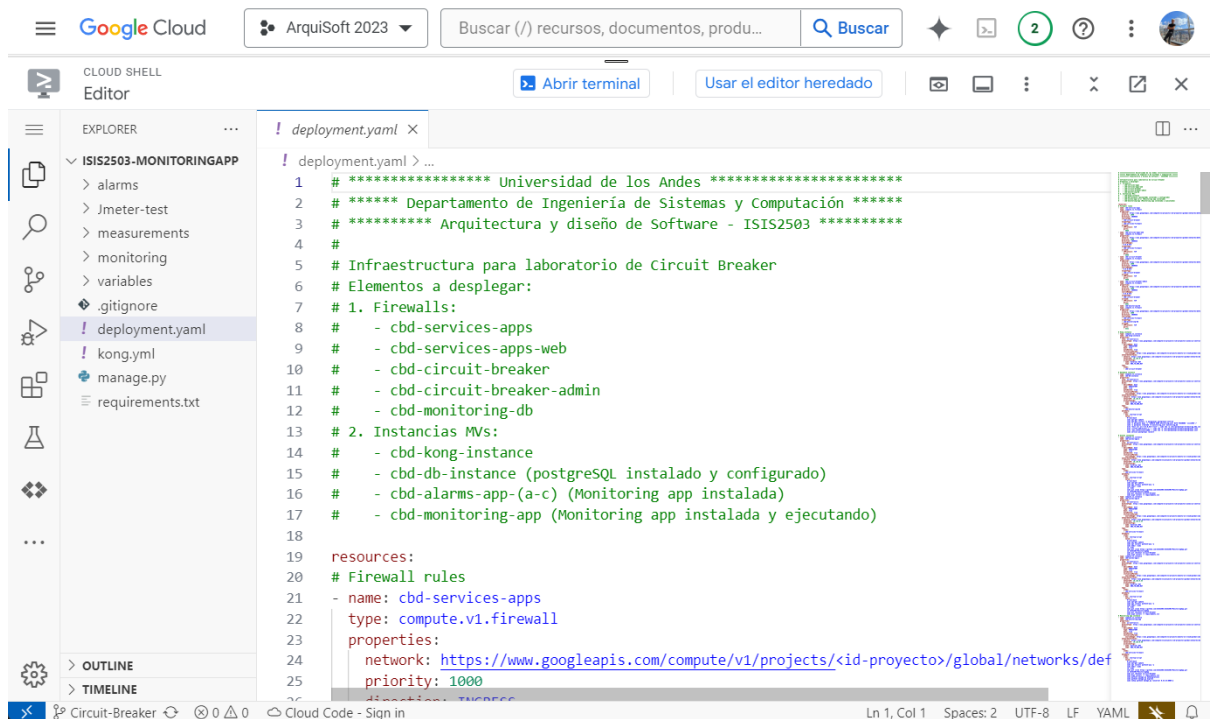
3. Abra el editor de GCP. Oprima el botón "Abrir editor" que aparece en la barra superior de CloudShell. Esto abrirá una interfaz muy similar a VS Code.



4. En el editor oprima "Abrir carpeta", seleccione "ISIS2503-MonitoringApp" y oprima OK. Esto abrirá el editor en la carpeta del repositorio de laboratorio en la rama "Circuit-Breaker" (por los comandos ejecutados en el punto anterior).



- Con el proyecto abierto en el editor, acceda al archivo "deployment.yaml". Este es el archivo de configuración que usará para desplegar la infraestructura. Observe el contenido del archivo.



- En el archivo se encuentran los recursos que se van a usar. Bajo la llave "resources" se listan estos recursos que para este laboratorio son de dos tipos: firewall e instancia de Compute Engine (MV). Cada tipo de recurso tiene propiedades que se pueden configurar. Por ejemplo, las reglas de firewall tienen "direction", "priority", "sourceTags", "targetTags", entre otras. Mientras que una instancia de Compute Engine puede tener "machineType", "disks", "networkInterfaces".

- **Firewall:**

Revise el punto 3.3 del [pre-laboratorio 4 – pruebas de carga](#), donde se crean las reglas de firewall. Para una regla de estas, la definición en el archivo de configuración puede ser la siguiente.

```
- name: cbd-monitoring-db
  type: compute.v1.firewall
  properties:
    network: https://www.googleapis.com/compute/v1/projects/<id-proyecto>/global/networks/default
    priority: 1000
    direction: INGRESS
    sourceTags:
      - cbd-services-firewall
    targetTags:
      - cbd-monitoring-db
    allowed:
      - IPProtocol: TCP
        ports:
          - 5432
```

Puede revisar el resto de las reglas que son un recurso de tipo "compute.v1.firewall".

- **Instancia de MV:**

Ahora, revise el punto 3.4 de ese pre-laboratorio donde se crea la máquina virtual para la base de datos. Todas las instrucciones indicadas se traducen en el recurso "cbd-db-instance" del archivo de configuración.

```
115 # Database instance
116 - type: compute.v1.instance
117   name: cbd-db-instance
118   properties:
119     zone: us-central1-a
120     machineType: https://www.googleapis.com/compute/v1/projects/<id-proyecto>/zones/us-central1-a/machineTypes/e2-micro
121     disks:
122       - deviceName: boot
123         type: PERSISTENT
124         boot: true
125         autoDelete: true
126         initializeParams:
127           sourceImage: https://www.googleapis.com/compute/v1/projects/ubuntu-os-cloud/global/images/ubuntu-2004-focal-v20240307b
128     networkInterfaces:
129       - network: https://www.googleapis.com/compute/v1/projects/<id-proyecto>/global/networks/default
130         networkIP: 10.128.0.52
131     tags:
132       items:
133         - cbd-monitoring-db
134     metadata:
135       items:
136         - key: startup-script
137           value: |
138             #!/bin/bash
139             sudo apt-get update
140             sudo apt-get install -y postgresql postgresql-contrib
141             sudo -u postgres psql -c "CREATE USER monitoring_user WITH PASSWORD 'isis2503';"
142             sudo -u postgres createdb -O monitoring_user monitoring_db
143             echo "host all all 0.0.0.0/0 trust" | sudo tee -a /etc/postgresql/12/main/pg_hba.conf
144             echo "listen_addresses=*" | sudo tee -a /etc/postgresql/12/main/postgresql.conf
145             echo "max_connections=2000" | sudo tee -a /etc/postgresql/12/main/postgresql.conf
146             sudo service postgresql restart
```

En este recurso se configura el tipo de máquina, el sistema operativo a usar, la interfaz de red (con una IP específica. Ej: 10.128.0.52), las etiquetas de red y un script de inicio con el que se ejecutan los comandos necesarios para la configuración y ejecución de PostgreSQL. Observe que en el script de inicio se configura el nombre de la base de datos, el usuario y la contraseña. Revise las demás instancias incluidas en el archivo que son recursos de tipo "compute.v1.instance". También tenga en cuenta que para este laboratorio la única instancia que tendrá IP externa será cbd-kong-instance y eso se define al agregar la propiedad

"networkInterfaces/accessConfigs", observe esa propiedad en el archivo. Para que el resto de VMs pueda acceder a internet se configura el router cbd-router con configuración NAT.

Nota: Todos los recursos definidos en este archivo tienen el prefijo "cbd-" que se refiere a "Circuit Breaker Deployment". Este prefijo permite la fácil identificación de los recursos en la consola de GCP.

7. En el archivo se encuentran varios enlaces que configuran redes, imágenes de sistemas operativos y tipos de máquinas. Algunos de estos enlaces son relativos a su proyecto, por lo que es necesario que edite este archivo para cambiar "<id-proyecto>" por el ID de su proyecto.

Ejemplo:

```
.../projects/<id-proyecto>/global/networks/default  
pasa a ser  
.../projects/arquisoft-2023-98786/global/networks/default
```

Debe cambiar **todas** las etiquetas <id-proyecto> que estén en el archivo "deployment.yaml". Se recomienda usar alguna herramienta para reemplazar texto. Para guardar dentro del editor de Cloud Shell puede usar Ctrl+S o Cmd+S.

8. Con el archivo de configuración listo, ejecute los siguientes comandos en CloudShell (puede volver al terminal con el botón "Abrir terminal" en la barra superior del editor de CloudShell). Asegúrese que el directorio activo sea el del repositorio ISIS2503-MonitoringApp.

```
>>> gcloud deployment-manager deployments create circuit-breaker-  
deployment --config deployment.yaml
```

Si el terminal se lo pide, habilite el API.

En caso de un despliegue exitoso podrá ver un listado de los recursos con "STATE" completado.

```
(arquisoft-2024-uniandes) x + - Editor
NAME: cbd-monitoring-app
TYPE: compute.v1.instance
STATE: COMPLETED
ERRORS: []
INTENT:

NAME: cbd-monitoring-db
TYPE: compute.v1.firewall
STATE: COMPLETED
ERRORS: []
INTENT:

NAME: cbd-services-apps
TYPE: compute.v1.firewall
STATE: COMPLETED
ERRORS: []
INTENT:

NAME: cbd-services-apps-web
TYPE: compute.v1.firewall
STATE: COMPLETED
ERRORS: []
INTENT:
javelino2311@cloudshell:~/ISIS2503-MonitoringApp (arquisoft-2024-uniandes) $
```

En caso de error, deberá resolverlo y luego eliminar el despliegue con el siguiente comando y volver a crear. Para resolver el problema puede pedir ayuda a los monitores o asistente de laboratorio. Puede eliminar un despliegue con el comando:

```
>>> gcloud deployment-manager deployments delete circuit-breaker-deployment
```

- Ahora puede dirigirse al listado de instancias de MVs. Puede observar que se encuentran las 6 instancias definidas en el archivo.

Instancias de VM

CREAR INSTANCIA

IMPORTAR VM

ACTUALIZAR

INSTANCIAS

OBSERVABILIDAD

PROGRAMAS DE LAS INSTANCIAS

Instancias de VM

Filtro

Ingresar el nombre o el valor de la propiedad

Estado	Nombre ↑	Zona	Recomendaciones	En uso por	IP interna
☐	cbd-alarms-app-a	us-central1-a			10.128.0.53 (nic0)
☐	cbd-alarms-app-b	us-central1-a			10.128.0.54 (nic0)
☐	cbd-alarms-app-c	us-central1-a			10.128.0.55 (nic0)
☐	cbd-db-instance	us-central1-a			10.128.0.52 (nic0)
☐	cbd-kong-instance	us-central1-a			10.128.0.51 (nic0)
☐	cbd-monitoring-app	us-central1-a			10.128.0.56 (nic0)

- Es importante resaltar que las instancias que ejecutan el script de inicio como db-instance o monitoring-app toman un tiempo en ejecutar las instrucciones iniciales. Probablemente debe esperar unos minutos para que esas instrucciones terminen de ejecutarse. **Se recomienda esperar 5 minutos.**

Notas:

- La instancia "cbd-monitoring-app" realiza automáticamente las migraciones a la base de datos, pero no inicia el servidor de Django. Esto significa que usted debe iniciarlo manualmente como en los laboratorios anteriores. Para esto

puede ingresar al SSH de la máquina, ubicarse en la carpeta del proyecto e iniciar Django usando los siguientes comandos.

```
>>> cd /labs/ISIS2503-MonitoringApp/  
>>> sudo python3 manage.py runserver 0.0.0.0:8080
```

En este laboratorio la única máquina virtual que tiene IP pública o externa es la que ejecuta Kong (cbd-kong-instance). Por esto, no será posible probar el funcionamiento de forma directa a las otras instancias como la de cbd-monitoring-app. Luego de que configure el servicio de Kong podrá probar el funcionamiento del resto de máquinas.

- Si quiere observar los logs de la ejecución del script de inicio puede ejecutar en la instancia el siguiente comando.

```
>>> sudo journalctl -u google-startup-scripts.service
```

- Existe la opción de ejecutar Django en segundo plano con el objetivo de no necesitar la terminal abierta. Para esto, puede reemplazar el comando de inicio de Django por el siguiente.

```
>>> sudo nohup python3 manage.py runserver 0.0.0.0:8080 &
```

Este comando hace uso de *nohup* y el signo *&*. El comando *nohup* o "no hang up" ejecuta procesos incluso si se cierra la terminal, y el signo *"&"* permite ejecutar en segundo plano un proceso dentro del terminal. Tenga en cuenta que al ejecutar los procesos de esta manera los logs quedan guardados en el archivo "nohup.out" en el directorio donde se ejecutó.

2.2. Configuración de los servicios de alarmas

- Con el despliegue automático, se crearon 3 instancias de alarmas: "cbd-alarms-app-(a,b,c)". Estas instancias ejecutaron un script de inicio que clonó el repositorio del laboratorio (en la ruta /labs/ISIS2503-MonitoringApp), cambió la rama a "Circuit-Breaker" e instaló las dependencias de Python. Sin embargo, no ejecutó las aplicaciones. Usted debe hacer esto manualmente. Para esto siga las siguientes instrucciones.

1. Conéctese a cada instancia de alarmas por SSH.
2. Ubíquese en el directorio del repositorio.

```
>>> cd /labs/ISIS2503-MonitoringApp/
```

3. Actualice el archivo **urls.py** del proyecto habilitar la ruta de la aplicación alarms en el proyecto monitoring. Para esto abra el archivo con el comando que le damos a continuación y descomente (quitar el #) de la línea `path("", include('alarms.urls'))` y comente (agregar un #) las siguientes rutas.

```
>>> sudo nano monitoring/urls.py
```

```

"""
from django.contrib import admin
from django.urls import include, path
from . import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('health/', views.health_check, name='health'),
    #path('', views.index),
    #path('', include('measurements.urls')),
    #path('', include('variables.urls')),
    path('', include('alarms.urls')),
]

```

4. Guarde los cambios que hizo en settings.py con “Ctrl+X”.
5. Ejecute la aplicación.

```
>>> sudo python3 manage.py runserver 0.0.0.0:8080
```

```

javelino2311@cbd-alarms-app-a:/labs/ISIS2503-MonitoringApp$ sudo python3 manage.py runserver 0.0.0.0:8080
Performing system checks...

System check identified no issues (0 silenced).
April 03, 2024 - 17:16:22
Django version 2.1.5, using settings 'monitoring.settings'
Starting development server at http://0.0.0.0:8080/
Quit the server with CONTROL-C.

```

6. Revise el código del proyecto (puede verlo en [GitHub](#)), en especial la sección de alarmas, y deduzca cómo se generan las alarmas. Genere por lo menos una alarma **después de configurar Kong**. Para esto debe:
 - i. Crear una variable a través de la aplicación web (cbd-monitoring-app).
 - ii. Crear una medición a través de la aplicación web (cbd-monitoring-app). La medición debe superar el límite especificado en el código que genera la alarma.
 - iii. Realizar una petición al endpoint que genera la alarma (puede usar Postman o el navegador).
- No cierre la conexión SSH ya que necesita los servicios ejecutándose para continuar con el laboratorio.

2.3. Configuración del servicio Kong

- Una vez la VM *kong-instance* se encuentre iniciada, conéctese a la máquina mediante la terminal SSH de Google (esto se explica en el laboratorio [Como crear una máquina virtual](#)) o a través de Putty (esto se explica en el siguiente [instructivo](#)). Una vez conectado a la máquina continúe con los siguientes pasos:
- En esta instancia se va a crear el contenedor de Kong, el cual se usará como bróker para implementar el circuit-breaker. Para esta implementación se configurará Kong de manera db-less (database less), lo que permitirá crear un archivo yaml para la configuración de servicios, rutas y upstreams. Los siguientes comandos se

deben ejecutar en la terminal de la instancia *kong-instance* para la instalación de Docker (más adelante en el curso veremos más detalladamente Docker)

```
>>> sudo apt-get update

>>> sudo apt-get install ca-certificates curl gnupg lsb-release -y

>>> sudo mkdir -m 0755 -p /etc/apt/keyrings

>>> curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo
gpg --dearmor -o /etc/apt/keyrings/docker.gpg

>>> echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs)
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

>>> sudo apt-get update

>>> sudo apt-get install docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin -y

>>> sudo groupadd docker

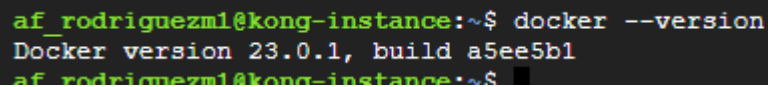
>>> sudo usermod -aG docker $USER

>>> newgrp docker
```

- Cuando finalice, ejecute el siguiente comando en terminal para validar la correcta instalación de Docker

```
>>> docker --version
```

- Debería aparecer como resultado el siguiente mensaje



```
af_rodriguezml@kong-instance:~$ docker --version
Docker version 23.0.1, build a5ee5b1
af_rodriguezml@kong-instance:~$
```

- En el siguiente enlace, encontrará el archivo [kong.yml](#) donde se define de manera declarativa la configuración del bróker. En la primera sección del documento encontrará la definición de los servicios, para configurar un servicio en Kong es necesario definir los siguientes campos
 - **host:** Dirección del host o nombre del upstream usado para el redireccionamiento de las peticiones.
 - **name:** Nombre del servicio.
 - **protocol:** Protocolo de comunicación usado con el upstream
 - **routes.name:** Nombre de las rutas
 - **routes.paths:** Path de enrutamiento usado por Kong para redireccionar peticiones
 - **routes.strip_path:** Flag usado para indicar si se desea eliminar el prefijo que se envía al upstream cuando un path hace match.
 - Para este caso se dividió el proyecto en dos hosts, un host dedicado a resolver peticiones del servicio monitoring y otro host dedicado a resolver peticiones del servicio alarms

```

3  services:
4    - host: monitoring_upstream
5      name: monitoring_service
6      protocol: http
7      routes:
8
9        - name: monitoring
10         paths:
11           - /
12         strip_path: false
13
14    - host: alarms_upstream
15      name: alarms_service
16      protocol: http
17      routes:
18
19        - name: alarms
20         paths:
21           - /alarms
22           - /alarmsValidate/
23         strip_path: false
24
  
```

- Continuando con la configuración del archivo, en la sección inferior se encuentra la definición de los upstreams. Los Upstreams representan un hostname virtual, que permite agrupar varias instancias o servicios (de la aplicación, no de Kong) y a su vez pueden ser usados como balanceadores de carga. Los siguientes atributos son usados para definir un Upstream en Kong:
 - name:** Nombre del Upstream. Este debe ser igual al del host de un servicio previamente definido.
 - targets:** Los targets son ips o hostnames, acompañados por un puerto de conexión, que permiten identificar una instancia del servicio de backend
 - targets.target:** Ip del host, acompañada del puerto
 - targets.weight:** Porcentaje para definir la distribución de esfuerzo entre las instancias del Upstream

```

24
25  upstreams:
26    - name: alarms_upstream
27      targets:
28        - target: <ip-privada-alarms-app-a>:8080
29          weight: 100
30        - target: <ip-privada-alarms-app-b>:8080
31          weight: 100
32        - target: <ip-privada-alarms-app-c>:8080
33          weight: 100
34
35    - name: monitoring_upstream
36      targets:
37        - target: <ip-privada-monitoring-app>:8080
38          weight: 100
39
  
```

- Actualice el archivo para apuntar a los targets esperados durante el laboratorio. Reemplace los <ip-privada-alarms-app-#> y <ip-privada-monitoring-app> por las IPs privadas (IP interna) de las instancias en GCP, estas deben ser las MVs creadas en el despliegue (con prefijo "cbd-"). Luego de esto copie el contenido del archivo y ejecute en consola el siguiente comando

```
>>> sudo nano kong.yml
```

- Copie el contenido del archivo presionando "Ctrl + v" y guarde los cambios con "Ctrl + x", "y", "Enter",
- Una vez haya copiado la información del archivo, ejecute el siguiente comando para desplegar un contenedor en Docker con la imagen de Kong.

```
>>> sudo docker network create kong-net
```

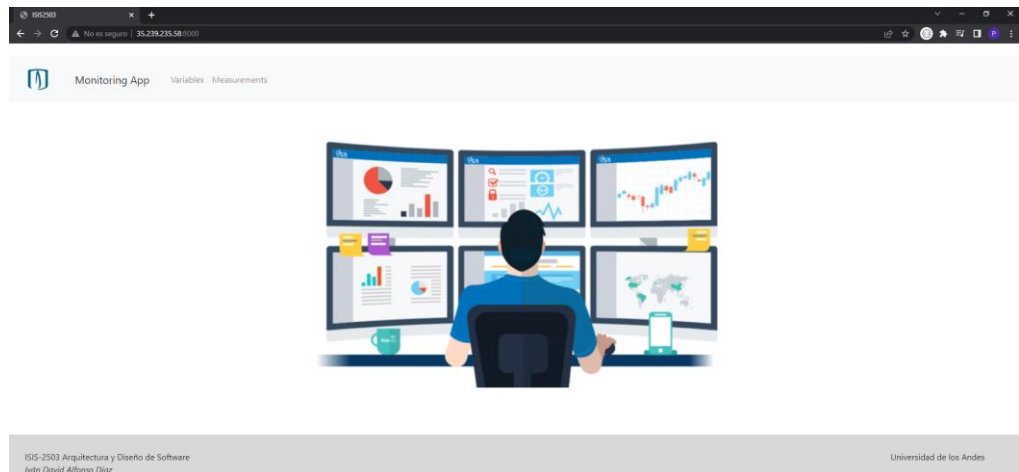
```
>>> sudo docker run -d --name kong --network=kong-net -v
"$(pwd):/kong/declarative/" -e "KONG_DATABASE=off" -e
"KONG_DECLARATIVE_CONFIG=/kong/declarative/kong.yml" -e
"KONG_PROXY_ACCESS_LOG=/dev/stdout" -e
"KONG_ADMIN_ACCESS_LOG=/dev/stdout" -e
"KONG_PROXY_ERROR_LOG=/dev/stderr" -e
"KONG_ADMIN_ERROR_LOG=/dev/stderr" -e "KONG_ADMIN_LISTEN=0.0.0.0:8001"
-e "KONG_ADMIN_GUI_URL=http://localhost:8002" -p 8000:8000 -p 8001:8001
-p 8002:8002 kong/kong-gateway:2.7.2.0-alpine
```

- Si la instalación de Docker fue correcta y el contenedor se desplegó correctamente ejecute el siguiente comando para validar el funcionamiento de Kong

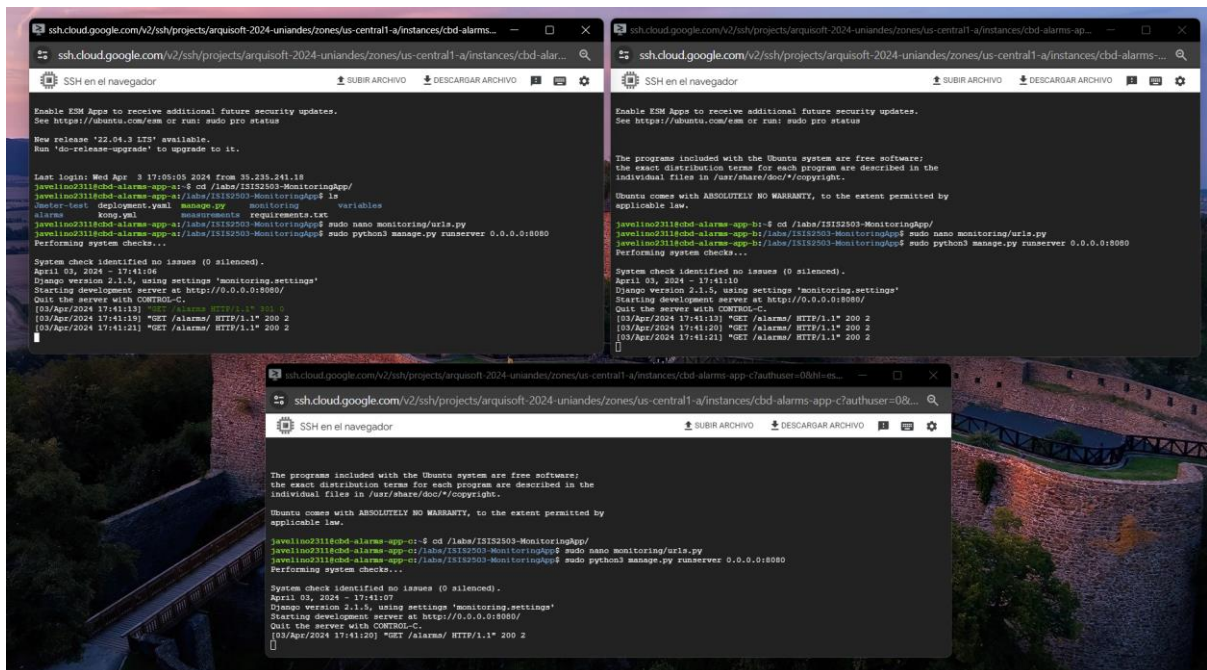
```
>>> docker ps
```

af_rodriguezml@kong-instance:~\$ sudo docker ps							
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES	
77bc2ffe2d22	kong/kong-gateway:2.7.2.0-alpine	"/docker-entrypoint..."	45 seconds ago	Up 41 seconds (healthy)	8003-8004/tcp, 0.0.0.0:8000-8002->8000-8002/tcp, :::8000-8002->8000-8002/tcp, 8443-8447/tcp	kong	
af_rodriguezml@kong-instance:~\$							

- Ingresa la dirección pública de la instancia acompañada del puerto 8000. Como podrá apreciar, Kong redirecciona las peticiones sin ruta al servicio de monitoring. De igual forma, puede navegar usando los botones de variables y measurements, esto ocurre debido a que en la definición de las rutas solo definimos estrictamente las rutas de alarmas y esto asignará las demás peticiones al servicio monitoring.



- Cree las variables *Temperature* y *Oxygen* y agregue varias medidas de measurements. Al menos una measurement debe ser mayor a 30 de value.
- Ahora dirijase a la ruta `/alarms` del servicio de kong. Debido a que el servicio de Kong funciona como Gateway y balanceador de carga, cuando definimos un Upstream Kong se encarga de distribuir las peticiones entre las instancias que están asignadas como Target. Tenga a la vista las 3 terminales de los servicios alarms y consulte repetidas veces la ruta `/alarms`



- La imagen anterior donde salen las consolas se puede apreciar como el servicio de Kong distribuye las peticiones entre las instancias de `alarms_upstream`, para este caso. Por defecto el algoritmo que usa Kong para balancear la carga es *Round-Robin*, pero puede ser usado otro al definirlo con el atributo *algorithm*

2.4. Activación de estados de salud y circuit-breaker

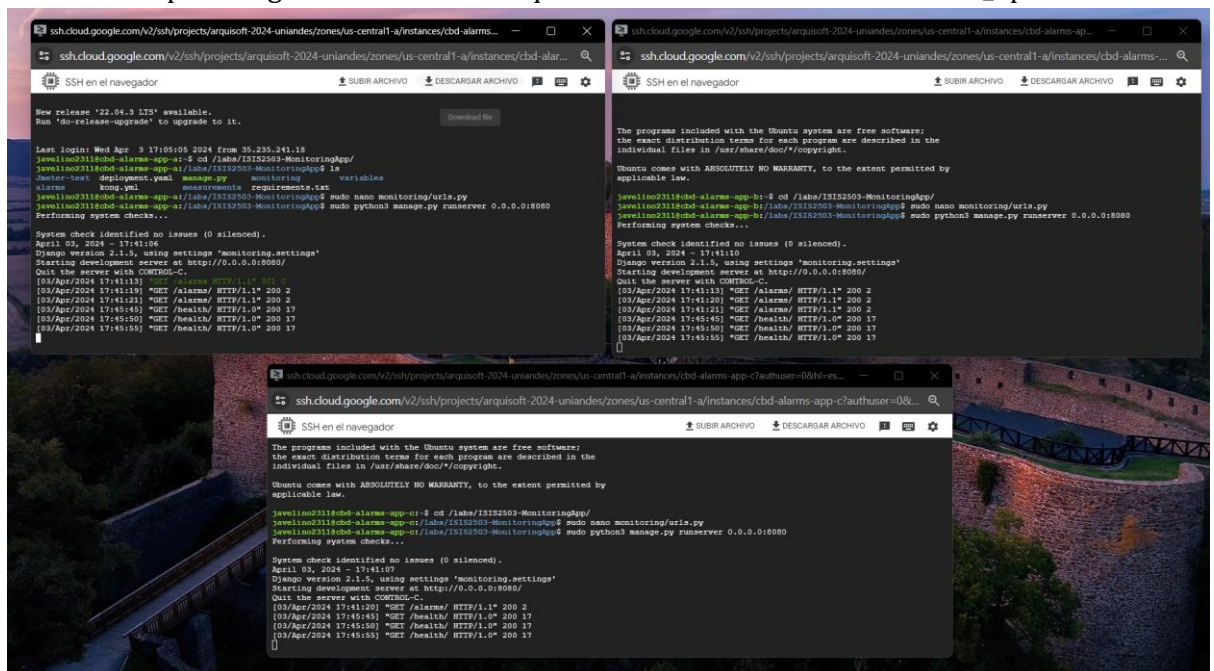
- Por el momento kong nos ha permitido enrutar y distribuir peticiones y cargar las peticiones entre un grupo de instancias. A continuación, configuraremos los estados de salud activos (active-health-checks) para identificar el estado de un grupo de instancias.
- Actualice nuevamente el archivo kong.yml descomentando (borre el #) las líneas de código que se encuentran en la definición del Upstream alarms_upstream. Los siguientes atributos son usados para definir los health-check
 - **threshold:** Es el porcentaje mínimo de instancias saludables de un Upstream
 - **active:** Permite definir health-checks de manera activa para identificar la salud de los servicios
 - **active.http_path:** Define el path al que apuntarán los health-checks
 - **active.timeout:** Tiempo máximo de espera de una petición antes de ser considerada fallida
 - **active.healthy.successes:** Número de solicitudes exitosas para que el servicio sea considerado saludable
 - **active.healthy.interval:** Tiempo en segundos entre una petición de validación y la otra
 - **active.unhealthy.tcp_failures:** Número de peticiones fallidas con los que considera un servicio no saludable
 - **active.healthy.interval:** Tiempo en segundos entre una petición de validación y la otra. Como podrá apreciar, tanto healthys como unhealthys pueden manejar intervalos con valores diferentes.

```
upstreams:
  - name: alarms_upstream
    targets:
      - target: <ip-privada-alarms-app-a>:8080
        weight: 100
      - target: <ip-privada-alarms-app-b>:8080
        weight: 100
      - target: <ip-privada-alarms-app-c>:8080
        weight: 100
    #healthchecks:
    #  threshold: 0
    #  active:
    #    http_path: /health/
    #    timeout: 0
    #    healthy:
    #      successes: 0
    #      interval: 0
    #    unhealthy:
    #      tcp_failures: 0
    #      interval: 0
```

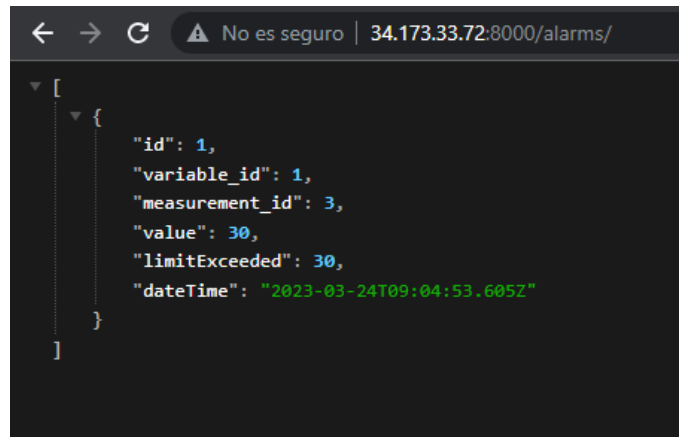
- Modifique el archivo para que tenga los siguientes datos en la validación de salud de los servicios

```
weight: 100
healthchecks:
  threshold: 50
  active:
    http_path: /health/
    timeout: 5
  healthy:
    successes: 2
    interval: 5
  unhealthy:
    tcp_failures: 2
    interval: 5
```

- Una vez tenga modificado el archivo, ejecute el siguiente comando para reiniciar kong
- ```
>>> sudo docker restart kong
```
- Cuando la instancia de Kong esté ejecutándose nuevamente, podrá ver en las tres terminales peticiones con el path /health/. Esta validación es la validación activa hecha por Kong sobre los servicios que se encuentran inscritos a alarms\_upstream.

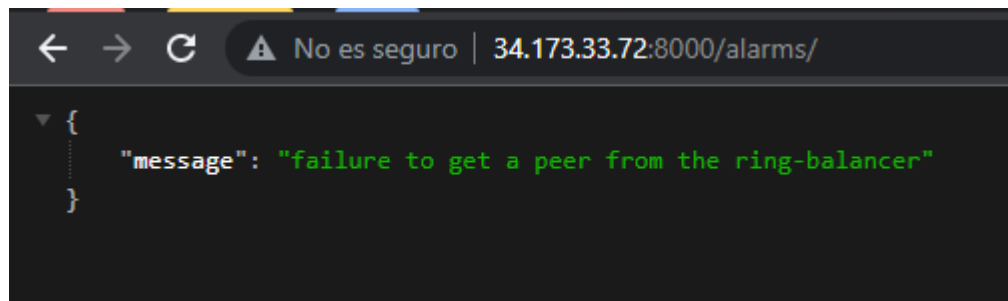


- El objetivo de un *Circuit Breaker* es enmascarar la degradación de un servicio, permitiendo seguir recibiendo peticiones, pero impidiendo que estas lleguen al servicio afectado y notificando al cliente que no podemos procesar su petición. En el archivo de configuración definimos un *threshold* de 50 por ciento, por lo que el servicio considerará que se encuentra degradado cuando menos del 50% de las instancias se encuentran saludables.
- Probaremos apagando el servicio de una de las instancias (Ctrl + C) y haciendo peticiones al servicio Kong. Al hacerlo podrá ver que el servicio sigue respondiendo con normalidad y las instancias siguen respondiendo a las peticiones, esto debido a que siguen estado sanas el 66% de las instancias del upstream.



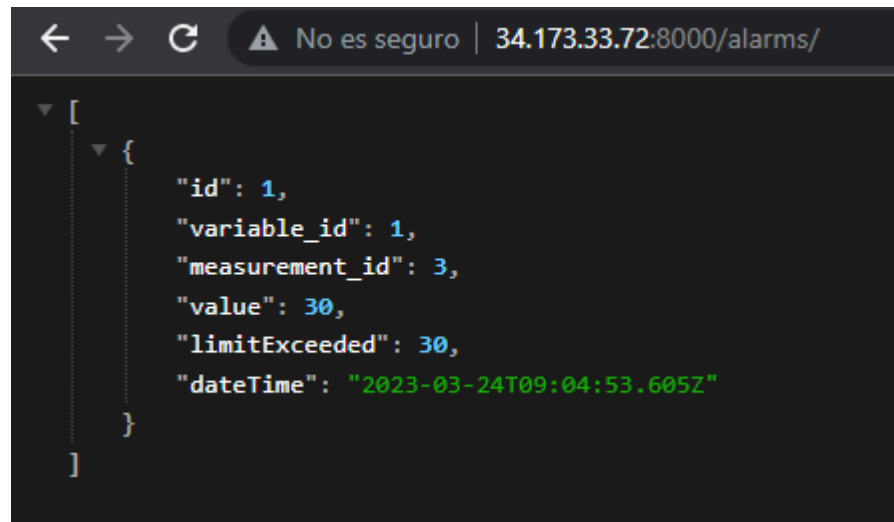
```
← → ↻ ⚠ No es seguro | 34.173.33.72:8000/alarms/
[
 {
 "id": 1,
 "variable_id": 1,
 "measurement_id": 3,
 "value": 30,
 "limitExceeded": 30,
 "dateTime": "2023-03-24T09:04:53.605Z"
 }
]
```

- Apague el servicio de una segunda instancia y pruebe haciendo peticiones al servicio Kong. Como en esta ocasión solo el 33% de las instancias del Upstream se encuentran sanas, el servicio Kong no envía peticiones a ninguna y adicionalmente responde con un mensaje de error al cliente. Este proceso no ocurre inmediatamente, mientras el servicio no ha cumplido con el número de peticiones necesarias para considerar el servicio no sano seguirá respondiendo peticiones. Si intenta varias veces podrá evidenciar que a la única instancia sana no le llegarán ninguna petición.



```
← → ↻ ⚠ No es seguro | 34.173.33.72:8000/alarms/
{
 "message": "failure to get a peer from the ring-balancer"
}
```

- Vuelva a encender uno de los servicios de alarmas, nuevamente el servicio no se restaurará inmediatamente hasta que no cumpla con el número de peticiones para considerarlo sano.



```
[
 {
 "id": 1,
 "variable_id": 1,
 "measurement_id": 3,
 "value": 30,
 "limitExceeded": 30,
 "dateTime": "2023-03-24T09:04:53.605Z"
 }
]
```

### 3. Entregables

Cree un documento con las evidencias de los siguientes puntos.

- Modifique el archivo *kong.yml* para agregar las condiciones de salud del Upstream del servicio *monitoring-app*. La configuración con la debe crear el nuevo servicio es
  - 35% de instancias sanas
  - 10 segundos de timeout
  - Health con 4 consultas con intervalos de 10 segundos.
  - Unhealth con 1 consulta e intervalos de 7 segundos.

La evidencia puede ser una captura del archivo resultante.

- Replique el servicio de *monitoring-app* para tener 3 instancias desplegadas (1 existente y 2 nuevas) y actualice el upstream de *monitoring* para redirigir tráfico a todas las 3 instancias.
  - Tenga en cuenta que las cuentas de facturación con créditos pueden tener límites sobre los recursos. En el caso de los créditos académicos (50 USD), se tiene un límite de 8 IPs activas, lo que limita las MVs a 8. En el caso de los créditos de la prueba gratuita de GCP (300 USD), se limitan las IPs a 4 solamente.
  - Adjunte una captura de las MVs creadas con el servicio ejecutándose.
  - Adjunte captura del archivo *kong.yml* con nuevas instancias configuradas.
- Pruebe el funcionamiento del circuit breaker deteniendo la ejecución del servicio progresivamente en las instancias hasta recibir el mensaje de la degradación del servicio.
  - Como evidencia, adjunte capturas de los terminales y la respuesta del servicio en el navegador por cada escenario.
- Indique si las tácticas implementadas permiten el cumplimiento de los ASRs 1 y 2 involucrados en el experimento.

- En caso afirmativo, explique cómo se beneficiaron los ASRs. De lo contrario, explique qué modificaciones podría hacer a la arquitectura (estilos o tácticas) para cumplir con los ASRs.
- En este laboratorio se introdujo la IaC. Compare el proceso de despliegue usando esta tecnología y no usándola (como en los laboratorios anteriores).
- Si el cliente pide el uso de IaC en el proyecto, analice las ventajas y desventajas del uso de IaC para ese contexto.
- Revisión de créditos disponibles. Siga [este video](#) como guía. En el documento agregue una captura de pantalla del estado de sus créditos actuales. Si tiene menos de 20 USD en créditos, avise **inmediatamente** a los monitores o el equipo docente.
- Entregue el documento en la actividad correspondiente en Bloque Neón. Puede consultar la rúbrica en esta misma actividad.

### Importante:

Si ya terminó el laboratorio puede eliminar la infraestructura para evitar costos adicionales. Puede eliminar un despliegue ejecutando el siguiente comando en la terminal de **CloudShell**.

```
>>> gcloud deployment-manager deployments delete circuit-breaker-deployment
```

O, si quiere retomar luego, puede detener las instancias del despliegue (las que tienen prefijo "cbd-") e iniciarlas cuando continúe con el laboratorio.